Backend with Django 4.0

Building Robust and Scalable Web Applications in Python

Simone Ricci

Lessons overview:

- **Lesson 1** (15/045 – 2 hours)**:**
  - Introduction to Backend Development and Django Framework
- **Lesson 2** (21/05 – 3 hours)**:**
  - Models and Database Management
- **Lesson 3** (22/05 – 2 hours)**:**
  - Views and URL Routing
- **Lesson 4** (28/05 – 3 hours)**:**
  - Templates and Forms
- **Lesson 5** (29/05 – 2 hours)**:**
  - Authentication and Authorization
- **Lesson 6** (4/06 – 3 hours)**:**
  - Django REST framework/ Flask
- **Lesson 7** (25/06 – 2 hours)**:**
  - Flask/Creating a Real-World Django Project ??

Introduction to Backend Development and Django Framework

Overview:

- Web basics:
  - HTTP, URL, Client-Server,..
  - Server-side programming
- Python basics
  - Variables, List, Classes,..
- Django basics
  - Django framework, Installation, Project structure,...
- Hello World application
  - The most basic Django project

Web Basics

HTTP, URL, Client-Server

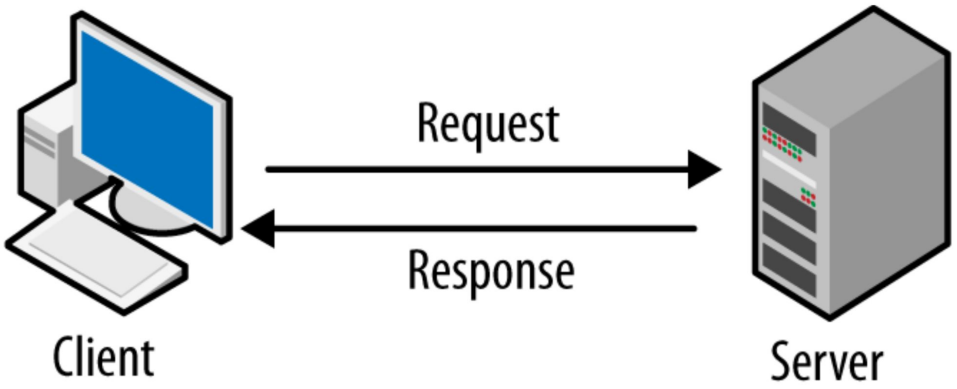
World Wide Web

In 1989 Tim Berners-Lee created the **HTTP** (Hypertext Transfer Protocol).

A network protocol is a set of rules for formatting and processing data.

HTTP is a **request-response** protocol that works in a **client-server** computing model.

Accessing a webpage, an initial “Request” is sent by the “Client” and a “Response” is sent back by a “Server.”



W3C

The World Wide Web Consortium (W3C) is the main international standards organization for the World Wide Web.

Founded in 1994 by Tim Berners-Lee.

It is made up of member organizations that maintain the development of standards for the World Wide Web:

HTML5, **CSS**, SVG, WOFF, the Semantic Web stack, XML, …

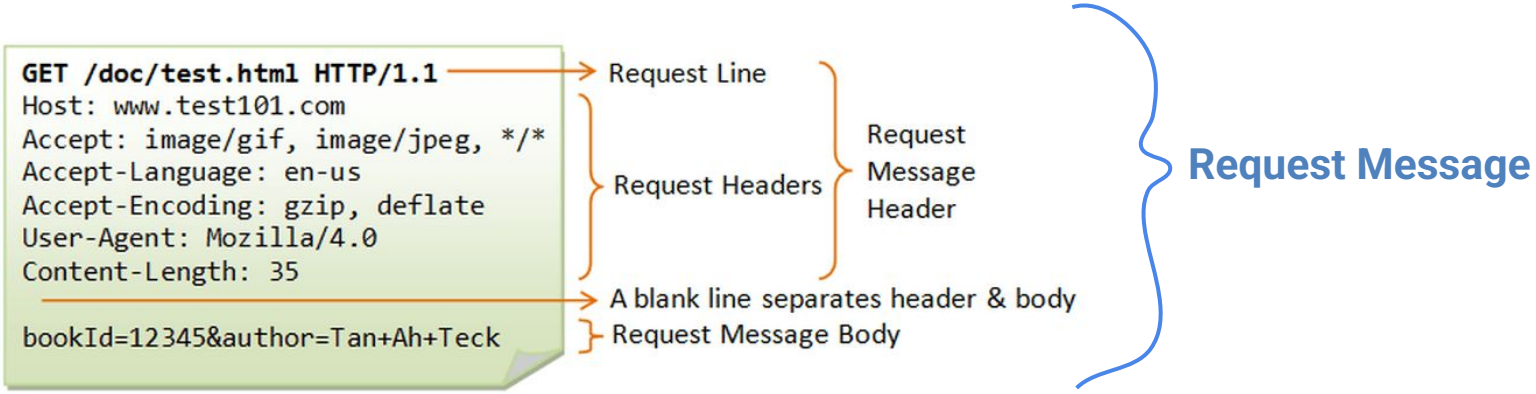
In 2023, W3C completed its transition to a public **non-profit organization**, strengthening its open standards mission.

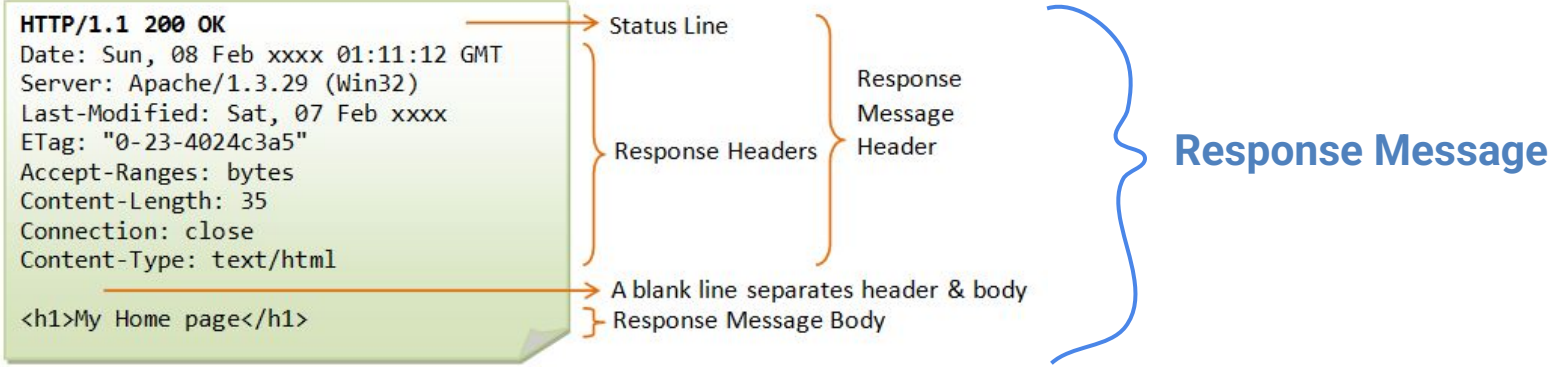
FunFact: https://www.w3.org/webat25/news/webfacts

HTTP

A request message consists of a **request line**, **headers**, and an optional **message body**.

The **request line** contains the method (such as GET, POST, PUT, DELETE), the URL of the resource, and the version of HTTP being used.

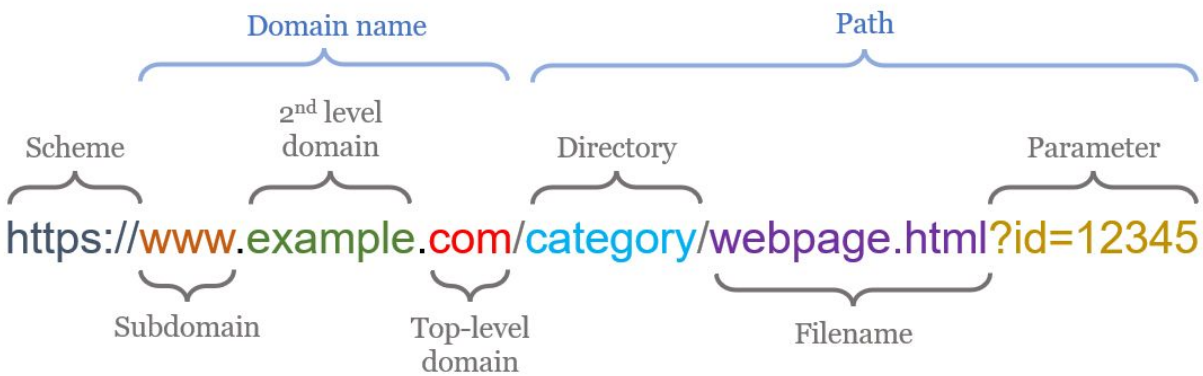




URL (Uniform Resource Locator)

A URL is a reference to a web resource that specifies its location on a computer network and a mechanism for retrieving it.

A typical HTTP URL could have the form of http://www.example.com/index.html
It indicates a **protocol** (http), a **hostname** (www.example.com), and a **filename** (index.html)



Every HTTP URL conforms to the syntax of a generic **URI** (Uniform Resource Identifier):

- URI = scheme ":" ["//" authority] path ["?" query] ["#" fragment]

HTTP Response

HTTP **Status Code** is a 3-digit numeric code that indicates the outcome of the HTTP request :

- Informational responses (100–199)
- Successful responses (200–299)
- Redirection messages (300–399)
- Client error responses (400–499)
- Server error responses (500–599)

Most frequent ones:

200 - OK: the request is successfully handled by the server.

301 - Moved Permanently: when the request is redirected to another URL permanently.

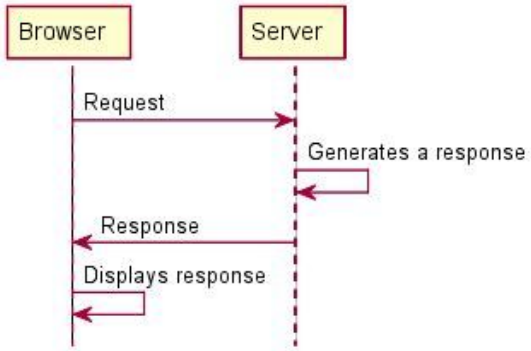
302 - Found: when the request is redirected to another URL temporarily.

404 - Not Found: when the server has no way of serving content corresponding to the indicated URL

500 - Server Error: when the request has generated an error on the server.

100 - Continue: which is used to tell the client “go ahead and send the request body”

Overview Client-Server Interaction



Here's a brief overview of how the web works:

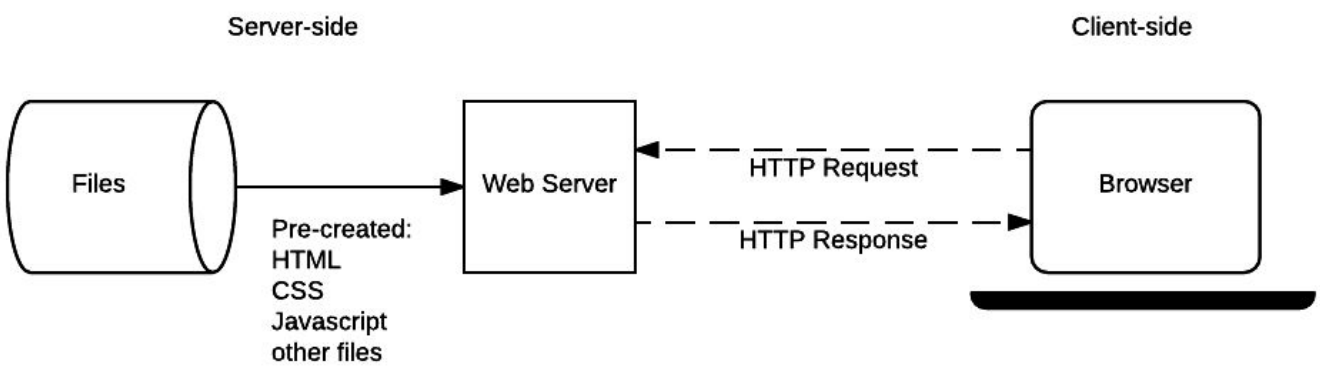
1. **Client Request**: The web browser on the client computer sends a request to the server. The request includes the URL of the resource being requested and the HTTP method being used.
2. **Server request processing**: The server receives the request and processes it by looking up the requested resource on its file system or database.
3. **Server Response**: The server sends an HTTP response back to the client, which includes the requested resource and any relevant metadata.
4. **Client response processing**: The client receives the response and renders the content in the web browser. This involves parsing the HTML markup, applying any stylesheets and JavaScript, and rendering the visual elements on the screen.
5. **Client additional requests**: If the web page includes links to other resources, such as images, scripts, or stylesheets, the client can send additional requests to the server to retrieve those resources.

Static sites

A “static site” is one that returns the **hard-coded content** (HTML, CSS, and JavaScript) from the server whenever a particular resource is requested.

The content of a static site is fixed and doesn't change unless **manually updated** by the developer.

They **lack interactivity** and can be **challenging to maintain** as the site grows.

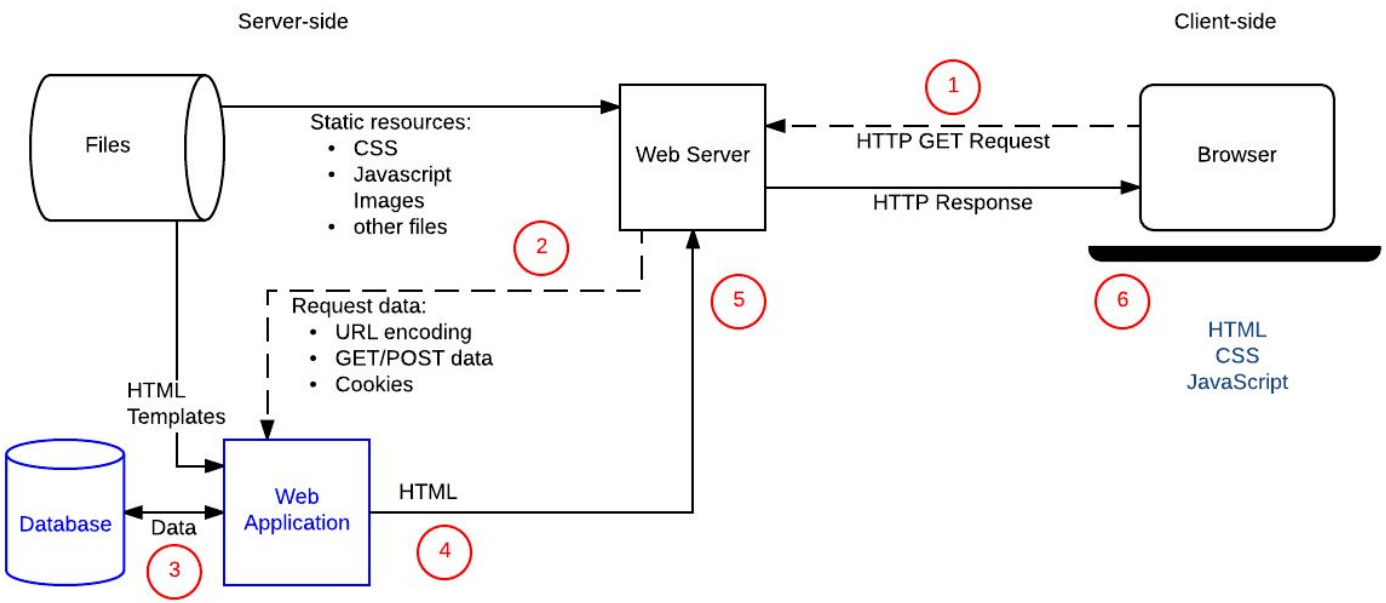


Dynamic sites

In a “dynamic site” some of the response content is generated **dynamically**, only when needed, by a server-side programming language like PHP, Python, or Ruby.

A dynamic site can return different data for the same URL based on information provided by the user or stored preferences.

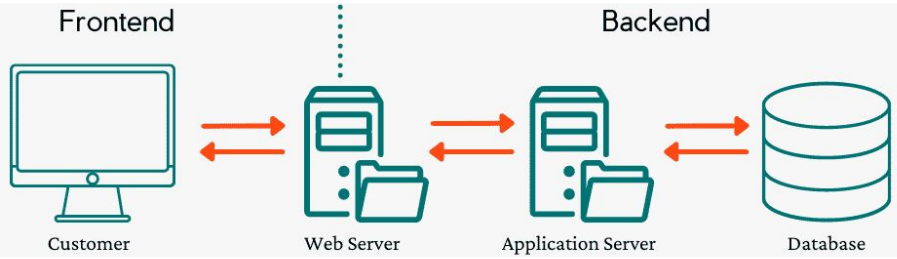
Most of the code runs on the server. Creating this code is known as "**server-side programming**" (or sometimes "**back-end scripting**").



Back-end vs Front-end

Front-end (F): the part of the system that the user interacts with.

Back-end (B): manages the data, logic, and processing behind the scenes.



Key differences:

- **User interface**: F is responsible for generating the UI
- **Programming languages:**
  - F: HTML, CSS, and JavaScript
  - B: PHP, Python, Ruby, or Java
- **Server vs. client-side**
  - F: the code runs on the user's device or web browser
  - B: the code runs on the server and communicates with F processing requests and sending responses
- **Security**
  - F: creates secure forms and handling input validation
  - B: manages user authentication, authorization, and data security

Server-side programming

It allows us to efficiently deliver information tailored for individual users and thereby create a **much better user experience**.

Companies like Amazon use server-side programming to:

- **construct search results** for products
- make **targeted product suggestions** based on client preferences and previous buying habits
- **simplify purchases**, etc.

Banks use server-side programming to **store account information** and allow only **authorized users** to view and make transactions.

Other services like Facebook, Twitter, Instagram, and Wikipedia use server-side programming to **highlight, share, and control access** to interesting content.

Common uses and benefits

Efficient storage and delivery of information:

Store the information in a database and dynamically construct and return HTML and other types of files.

Customized user experience:

Store and use information about clients to provide a better user experience.

Controlled access to content:

Restrict access to authorized users and serve only the information that a user is permitted to see.

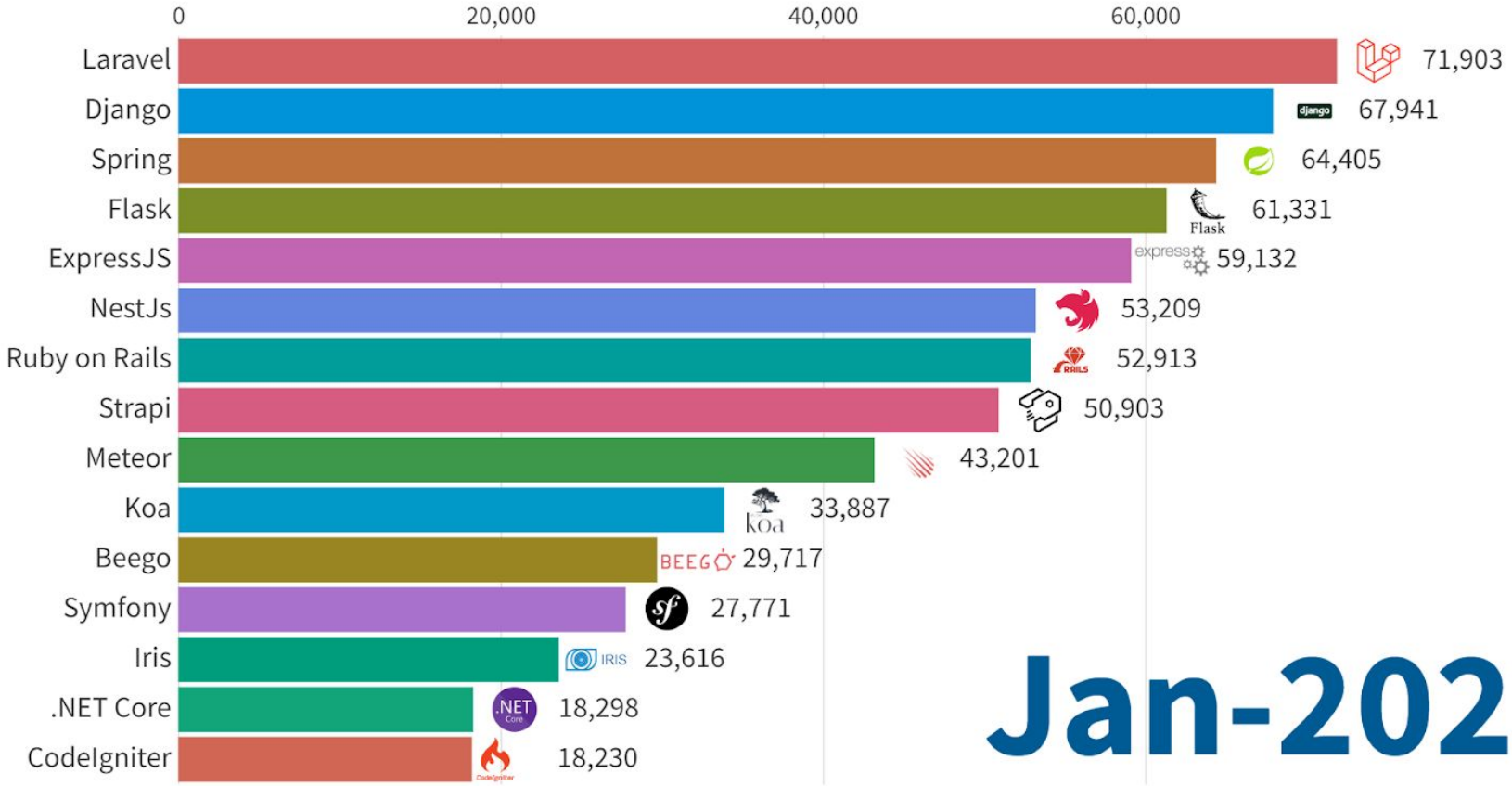
Store session/state information:

Use of sessions. A mechanism that allows a server to store information on the current user of a site and send different responses based on that information.

Notifications and communication:

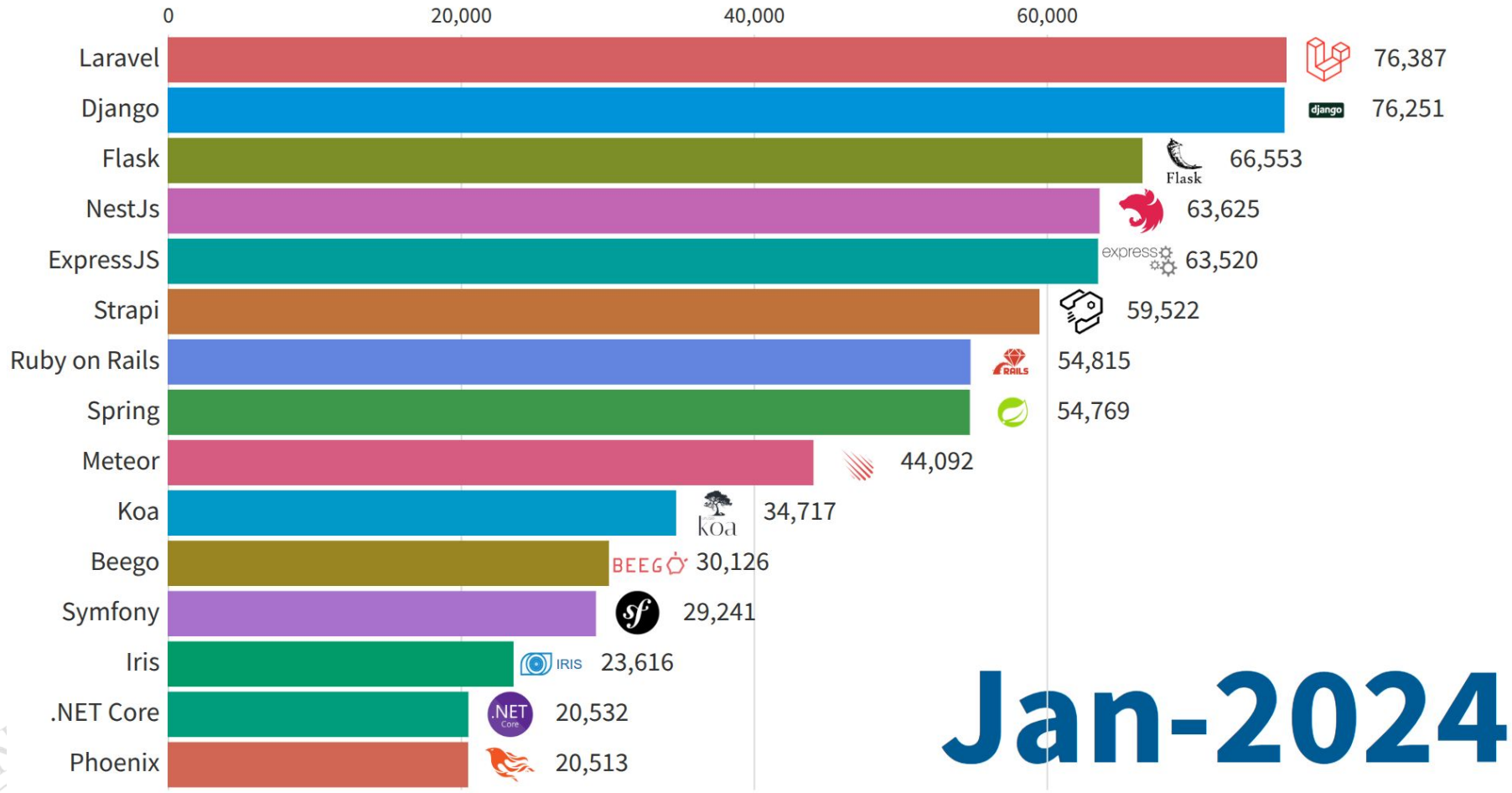
Servers can send general or user-specific notifications through the website itself or via email, SMS, instant messaging, video conversations, or other communications services.

Most Popular Backend Frameworks

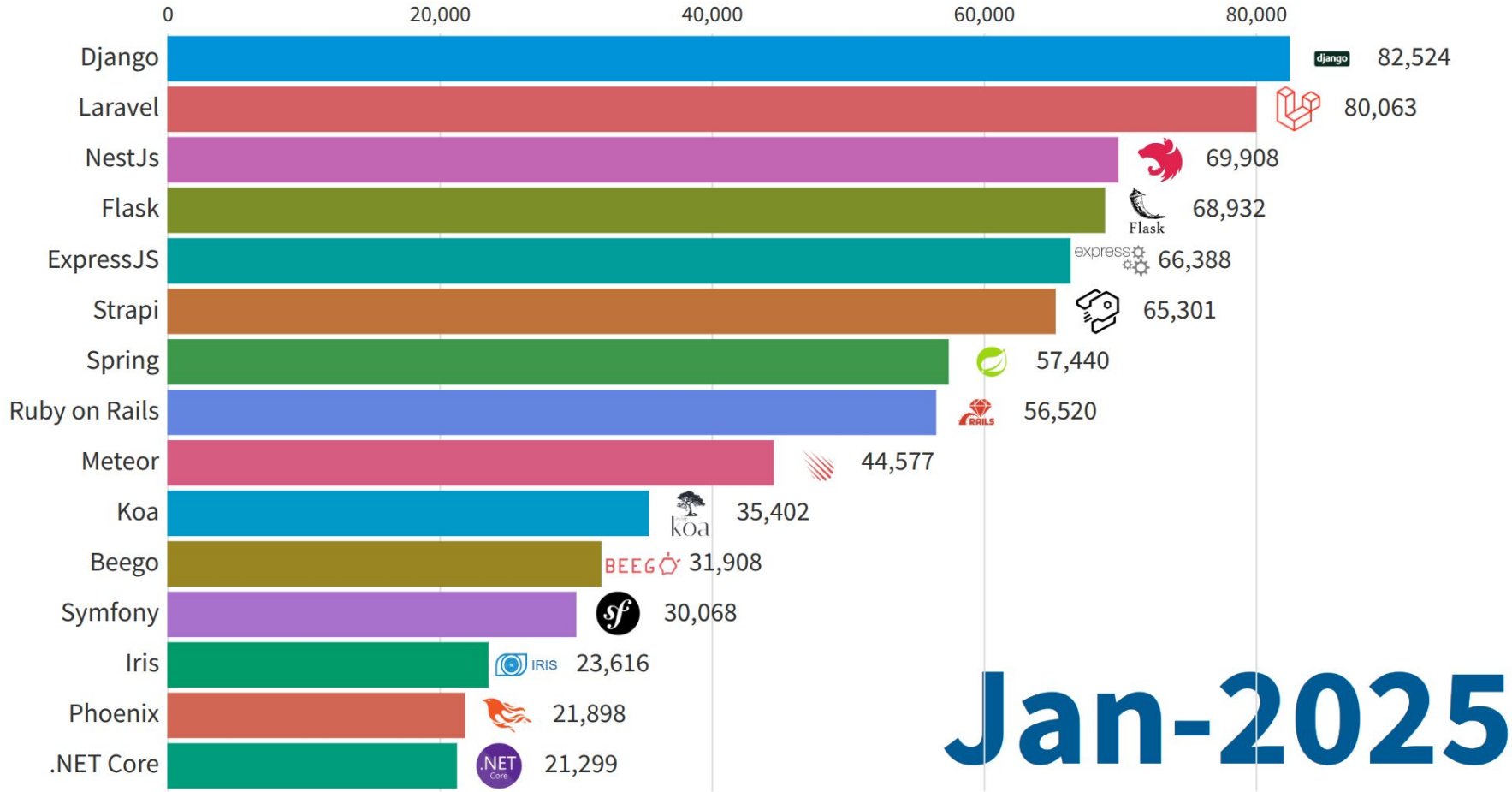


Jan-2023

Most Popular Backend Frameworks



Most Popular Backend Frameworks



Python Basics

Variables, Lists, Loops, Functions and Classes

Python

Python is a high-level, **interpreted** programming language that is widely used for a variety of tasks such as web development, data analysis, machine learning, and more.

It was created in the late 1980s by **Guido van Rossum** and has since become one of the most popular programming languages in the world.

It appears to have been developed in December 1989 as a side project to keep him busy over the Christmas holidays.

One of the reasons Python is so popular is its **simple** and **intuitive** syntax, which allows developers to write code **quickly** and easily.

It contains easter eggs: **import this** or **import antigravity**

Python key features

Syntax: Python has a clean and simple syntax. It is often described as being "executable pseudocode", meaning that it reads like plain English.

Dynamic typing: Python is dynamically typed, meaning that the data type of a variable is determined at runtime, rather than at compile time.

Interpreted: Python is an interpreted language, which means that it is executed directly by the interpreter, rather than being compiled into machine code. This allows for faster development and testing cycles, as well as easier portability across different platforms.

Strong standard library: Python has a large and comprehensive standard library that includes many useful modules for a wide range of tasks, such as working with files, networking, web development, and more.

Object-oriented: Python is an object-oriented language, which means that it is designed to work with objects and classes. This makes it easy to organize code into reusable and modular components.

Indentation

It refers to the **spaces** at the beginning of a code line.

Where in other programming languages the indentation in code is for readability only, the indentation in Python is very important.

Python uses indentation to **indicate a block of code.**

The number of spaces is up to the programmer, the most common use is **four**, but it has to be <u>at least one</u>.

```
if 5 > 2:
  print("Five is greater than two!")
```

Comments

They can be used to explain Python code.

They can be used to *make the code more readable*.

Comments can be used to prevent execution of some code lines.

Comments starts with a # or between """.

```
#This is a comment
print("Hello, World!")
"""
This is a comment
written in
more than just one line
"""
print("Hello, World!")
```

Variables

Variables are used to store values that can be used later in the program.

Python variables do not need to be declared with any particular data type.

The **interpreter** automatically identifies the data type based on the value assigned to the variable.

A variable can be casted to another type.

You can get the data type of a variable with the type() function.

```
x = 5
y = "John"
print(x)
print(y)
```

```
x = str(3)    # x will be '3'
y = int(3)    # y will be 3
z = float(3)  # z will be 3.0
```

```
print(type(x))
print(type(y))
```

Lists

Lists are a fundamental data structure in Python.

Lists are **mutable**, meaning that their contents can be modified after they are created.

A list can hold multiple values of any data type.

List items are indexed, the first item has index [0], the second item has index [1]

```
my_list = [1, 2, 3, "four", "five"]

print(my_list[0])   # prints 1
print(my_list[-1])  # prints five
```

A list can be accessed with a range of index numbers [2:4]

To add an item to the end of the list, use the **append()** method:

The **remove()** and **pop()** methods removes an specific item.

Tuples

A tuple is an ordered, immutable collection of objects

Tuples are **immutable**: once a tuple is created, its contents cannot be changed.

Tuples are defined using round brackets **()**.

Tuples are **faster than lists** for accessing elements, because they are stored in memory as a single block of data.

```
# Creating a tuple
my_tuple = (1, 2, 3, "four")

# Accessing elements of a tuple
print(my_tuple[0]) # Output: 1
print(my_tuple[3]) # Output: "four"

# Tuples are immutable
# This will raise a TypeError
my_tuple[0] = 10
```

Dictionary

In Python, a dictionary is **a collection of key-value pairs**, where each key is unique and maps to a corresponding value.

```
ages = {'Simone': 29, 'Emma': 26, 'Filippo': 32}
```

You can access the values in a dictionary by using the keys as indices:

```
print(ages['Simone'])  # Output: 29
```

You can also add or modify key-value pairs in a dictionary by assigning a value to a new or existing key:

```
ages['Simone'] = 30
ages['Filippo'] = 33
```

Loops

Python has a variety of looping structures, including **for** and **while** loops.

Loops in Python are used to execute a block of code repeatedly based on some condition.

A **for** loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string).

```
my_list = [1, 2, 3, "four", "five"]

for i in my_list:
    print(i)

i = 0
while i < len(my_list):
    print(my_list[i])
    i += 1
```

If-Else

if and **else** are used to create conditional statements.

The **if** statement checks whether a condition is **true**, and if it is, executes the code inside the block.

If the condition is **false**, the code inside the **else** block is executed instead.

The **elif** statement allows you to check multiple conditions, and execute different code depending on which condition is true.

```
num = 5

if num > 0: #condition1
    print(num, "is positive")
elif num < 0: #condition2
    print(num, "is negative")
else: # neither condition1 nor condition2 is true
    print(num, "is zero")
```

List Comprehension

List comprehension offers a **shorter syntax** when you want to create a new list based on the values of an existing list.

newlist = [*expression* **for** *item* **in** *iterable* **if** *condition* == True]

The return value is a new list, leaving the old list unchanged.

The condition is like a *filter* that only accepts the items that evaluate to True.

The iterable can be any iterable object, like a list, tuple, set etc.

The expression is the current item in the iteration, but it is also the outcome, which you can manipulate before it ends up like a list item in the new list.

```
newlist = [x for x in fruits if x != "apple"]
```

Functions

In Python, a function is a block of reusable code that performs a specific task.

Functions help in modularity, code reuse, and maintainability of the code.

They can take arguments and returns values.

```
def function_name(parameters):
    # Function body
    return value

def greet(name):
    print(f"Hello, {name}!")

greet("Simone")  # prints "Hello, Simone!"
```

***args** and ****kwargs** allow you to pass multiple arguments or keyword arguments to a function.

Functions parameters

Function parameters are passed by **reference**.

The parameter inside the function is actually a **reference to the same object in memory** as the argument that was passed in.

This is true for all types of objects in Python, including integers, strings, lists, and other objects.

The function can modify the object in place, and those modifications will be visible outside the function!

```
def modify_list(lst):
    lst.append(4)
    print("Inside function:", lst) #prints [1, 2, 3, 4]

my_list = [1, 2, 3]
modify_list(my_list)
print("Outside function:", my_list) # prints [1, 2, 3, 4]
```

Lambda Function

A lambda function is a small anonymous function.

A lambda function can take any number of arguments, but can only have one expression.

```
lambda arguments : expression
```

```
x = lambda a : a + 10
print(x(5))
```

```
def myfunc(n):
  return lambda a : a * n

mydoubler = myfunc(2)
mytripler = myfunc(3)
```

Annotation

Function annotations are arbitrary python expressions that are associated with various part of functions.

These expressions are evaluated **at compile time** and have no life in python's runtime environment.

Python does not attach any meaning to these annotations. They are interpreted by third party libraries, for example, **mypy**.

```
def incrementa(n: int) -> int:
    return n + 1
```

```
conto: int = 0
```

Classes

In Python, a class is a blueprint for creating objects that have certain properties (**attributes**) and can perform certain actions (**methods**).

Classes are defined using the class keyword, followed by the class name.

The __init__ method is a special method that is called when an object of the class is created. It is used to **initialize** the attributes of the object.

self parameter refers to **the object itself**. It is used to access the attributes and methods.

```python
class ClassName:
    def __init__(self, attributes):
        # Initialize attributes
        print(attributes)

    def method(self, parameters):
        # Method body
        print(parameters)

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def greet(self):
        print(f"Hello, my name is {self.name} and I am {self.age} years old.")

simone = Person("Simone", 30)
simone.greet()  # prints "Hello, my name is Simone and I am 30 years old."
```

Private Methods

A class is an example of **encapsulation** as it encapsulates all the data that is member functions, variables, etc.

In some scenarios we need to put restrictions on some methods of the class so that they can neither be accessed outside the class nor by any subclasses.

```
# Creating a Base class
class Base:

    # Declaring public method
    def fun(self):
        print("Public method")

    # Declaring private method
    def __fun(self):
        print("Private method")

# Creating a derived class
```

```
class Derived(Base):
    def __init__(self):

        # Calling constructor of
        # Base class
        Base.__init__(self)

    def call_public(self):

        # Calling public method of base class
        print("\nInside derived class")
        self.fun()

    def call_private(self):

        # Calling private method of base class
        self.__fun()
```

name mangling: any identifier of the form **__name** (at least two leading underscores or at most one trailing underscore) is replaced with **_classname__name**

!!

Private Variables

Private variables are **not truly private** in the sense that they cannot be accessed at all, but rather they are meant to *signal to other developers* that these variables should not be accessed directly.

A single **underscore _** at the beginning of an attribute's name tells other developers that the attribute is intended to be **private.**

Name mangling (__) changes the name of the variable in a way that makes it harder to guess or access from outside the class.

```
class MyClass:
    def __init__(self):
        self.__private_var = 10

obj = MyClass()
# Accessing name-mangled private variable (not recommended)
print(obj._MyClass__private_var)
```

Methods vs Functions

Both methods and functions are used to **encapsulate a block of code** that performs a specific task.

Methods are functions that are defined within a class.
They are **called on an instance of that class**.

Functions, on the other hand, are standalone blocks of code that can be **called from anywhere** in the program.

Methods have access to the data and attributes of the object they are called on, while functions do not have this access.

Inheritance

Inheritance is a way of creating a new class by **inheriting the properties and methods** of an existing class (the parent class or base class).

The new class (the child class or derived class) can add new attributes and methods, or **override** the existing ones.

The child class can access the attributes and methods of the parent class using the **super()** function.

```
class Student(Person):
    def __init__(self, name, age, major):
        super().__init__(name, age)
        self.major = major

    def describe(self):
        print(f"I am studying {self.major}.")

emma = Student("Emma", 26, "Nutritionism")
emma.greet()    # prints "Hello, my name is Emma and I am 26 years old."
emma.describe() # prints "I am studying Nutritionism."
```

Decorator

A decorator takes a function, extends it and returns. A function can return a function!

Decorators are common and can be simplified. While it does exactly the same, it's just cleaner code.

Python can simplify the use of decorators with the **@ symbol.**

```
@hello
def name():
    print("Alice")

if __name__ == "__main__":
    name()
```

```
def hello(func):
    def inner():
        print("Hello ")
        func()
    return inner

def name():
    print("Alice")


obj = hello(name)
obj()
```

Python Exceptions

An **exception** is an event that disrupts the normal flow of a program—essentially, an error that occurs during execution.

Python "raises" an exception, which you can then **catch** and **handle** so your program can recover or at least fail gracefully.

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Oops! You can't divide by zero.")
else:
    # Runs only if no exception was raised
    print("Division successful:", result)
finally:
    # Always runs, exception or not
    print("Cleaning up resources…")
```

```
try:
    my_dict = {}
    print(my_dict["missing_key"])
except Exception as e:
    print(f"Caught a {type(e).__name__}: {e}")
```

This will show you both the type (**KeyError**) and the message (e.g. '**missing_key**').

You can also define custom exception classes by subclassing **Exception**.

You can trigger an exception yourself with **raise.**

Packages

They are **collections of modules and sub-packages.**
A package is simply a directory that contains a special file called **__init__.py.**

Python packages are used to make code more organized, maintainable, and reusable.

Python comes with a number of built-in packages but third-party packages are available on the **Python Package Index (PyPI)**. They can be easily installed using tools like pip.

To import a package in Python, you can use the **import** statement followed by the name of the package: import numpy

For a specific module or function from the package use **from**: from numpy import array

Note that you can also use an **alias** for the package or module with **as** keyword:
import numpy as np
from numpy import array as arr

Django Framework

Introduction to server-side programming in Python

Web framework

A “web framework” is a **collection of tools** that abstract away much of the difficulty inherent in web development.

Most websites need the **same basic functionality**: the ability to connect to a database, set URL routes, display content on a page, handle security properly, and so on.

Django inherited Python’s “**batteries-included**” approach and includes out-of-the box support for:

- user authentication
- database models, forms, URL routes, and templates
- admin interface
- security and performance upgrades
- support for multiple database backends
- testing

Django benefits

Fast development process:
Django provides a lot of built-in features and functionality that can speed up the development process. (**ORM**, **Templates, Forms**, etc)

Built-in security features:
Django provides many security built-in features that make it easy to create secure web applications. Common vulnerabilities such as **CSRF** (Cross-Site Request Forgery), **SQL injection**, and XSS (Cross-Site Scripting) can be handle easily.

Scalability:
Django is designed to **handle high traffic loads**. Django can handle large amounts of traffic and can be easily scaled up by adding more resources to the server.

Versatility:
Django is a versatile framework that can be used for a wide range of web applications. Django's **modular structure** and reusable components make it easy to build and maintain complex web applications.

Large community:
Django has a large and active community of developers who contribute to its development. It ensures that Django will continue to be updated and improved.

Django Installation

Django is a Python-based web framework: it relies on Python's libraries and packages to work.

A Python version need to be installed on your system.

Anaconda comes to our help.

Anaconda is an open-source distribution of the Python and R programming languages with cross-platform support. (https://www.anaconda.com/)

It has a **Conda package manager**, which makes it easy to manage and install packages, as well as create and manage virtual environments for different projects.

After installed Anaconda, we can create a new python environment with the **terminal** command:

conda create -n django python=3.12

To **activate** the environment: *conda activate django*

Django is hosted on the Python Package Index (**PyPI**), a central repository for most Python packages.

With the right environment activated, install Django with: *pip install Django==5.2.1*

Django new project

A Django project can have almost any **name** but we will use “django_project”.

To create a new Django project use the command:
django-admin startproject django_project

This command creates the fundamental files for a django application with this structure:

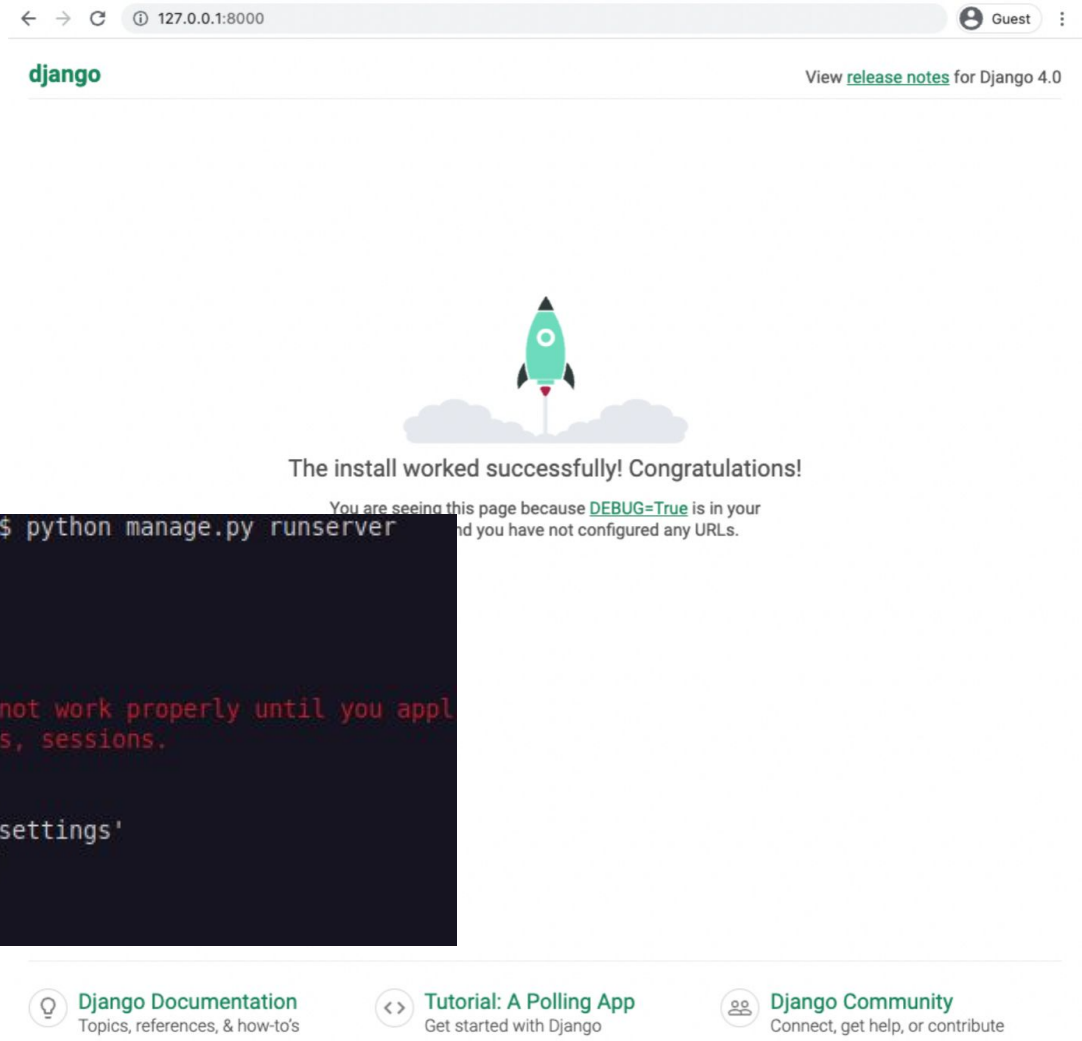
```
django_project/
    ├── django_project
    │   ├── __init__.py
    │   ├── asgi.py
    │   ├── settings.py
    │   ├── urls.py
    │   └── wsgi.py
    └── manage.py
```

To check the creation we can start the Django’s internal web server with *runserver* command: *python manage.py runserver*

Django default page

There are 18 unapplied migration(s).

They are related to some changes in the models.




```
(django) sricci@tarma:~/Desktop/django/django_project$ python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you appl
y the migrations for app(s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
March 29, 2023 - 10:59:32
Django version 4.1.7, using settings 'django_project.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.
```

Default project structure

Django has added a **django_project directory** and a **manage.py** file. Within django_project are five files:

- **__init__.py :** indicates that the files in the folder are part of a Python package.
- **asgi.py :** allows for an optional Asynchronous Server Gateway Interface to be run
- **settings.py :** controls Django project's settings
- **urls.py :** tells Django which pages to build in response to a browser or URL request
- **wsgi.py** : Web Server Gateway Interface which helps Django serve our eventual web pages.

The **manage.py** file is used to execute various Django framework commands such as running the local web server or creating a new app.

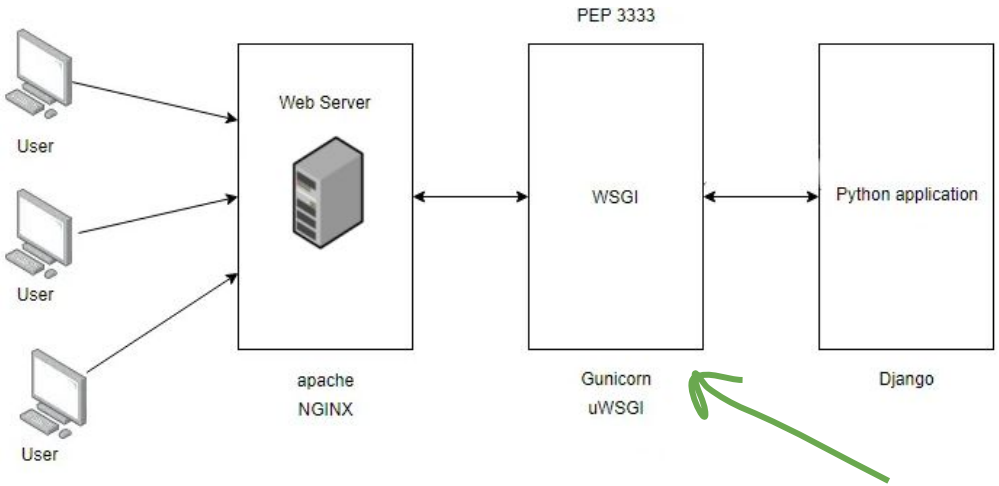
```
django_project/
    ├── django_project
    │   ├── __init__.py
    │   ├── asgi.py
    │   ├── settings.py
    │   ├── urls.py
    │   └── wsgi.py
    └── manage.py
```

Web Server Gateway Interface (wsgi.py)

WSGI is a specification that describes the communication between web servers and Python web applications or frameworks.

A web server uses WSGI to pass a received request from a client to Django application. The response is then passed back to the web server using WSGI.

WSGI allows developers to use a variety of web servers with their Django application without needing to modify the application code (**flexibility**).



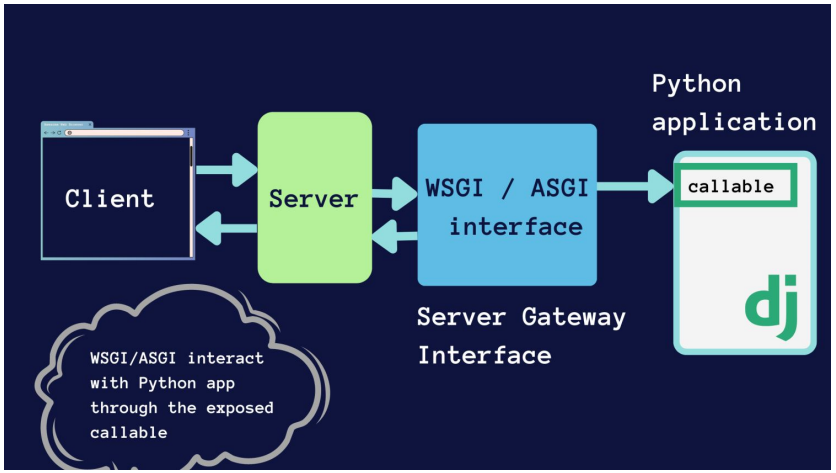
Asynchronous Server Gateway Interface (asgi.py)

ASGI is a specification for asynchronous web servers and web applications in Python.

It allows a more **efficient and scalable handling of HTTP** requests and responses, particularly for long-lived connections such as WebSockets or HTTP/2.

ASGI supports **asynchronous I/O**, allowing a server to handle multiple requests simultaneously without having to spawn multiple threads or processes.

It can improve performance and scalability, particularly for applications with high levels of concurrent connections.



URL (urls.py)

The URL file generated by Django is a Python module that **maps URLs to Views** in a Django application.

The **urls.py** file defines a list of URL patterns using regular expressions, which are then matched against incoming requests.

```
HTTP Request -> URL -> Django combines database, logic, styling -> HTTP Response
```

If it doesn't find a match, it will return a **404 error**.

```python
from django.contrib import admin
from django.urls import path

urlpatterns = [
    path('admin/', admin.site.urls),
]
```

Django Settings (settings.py)

The **settings.py** file is a Python module that contains various configuration settings for a Django project.

It is organized into several section (not only these!):

- **SECRET_KEY**: This setting is used to provide cryptographic signing for various aspects of the Django project (cookies and sessions). It should be kept secret and not shared publicly.
- **DEBUG**: This setting determines whether or not the Django project is in debug mode.
- **ALLOWED_HOSTS**: This setting specifies a list of hostnames that are allowed to access the Django project. It is used to prevent **unauthorized access**.
- **INSTALLED_APPS**: This setting specifies a list of applications that are installed in the Django project.
- **DATABASES**: This setting specifies the configuration for the database that the Django project will use (MySQL , PostgreSQL, etc).
- **STATIC_URL**: These settings specify the URL for static files, such as CSS and JavaScript files.
- **TEMPLATES**: A list of template engines that are installed in the Django project, along with their configurations.

manage.py

It is a **command-line utility** that allow developers to perform various tasks related to their Django project.

Some common tasks can be performed:

- **runserver**: Starts the development server and listens for incoming requests.
- **createsuperuser**: Creates a new superuser account that can be used to access the Django admin interface.
- **startapp**: Generates a new Django app, including a default directory structure and some basic files.
- **shell**: Opens an interactive Python shell with access to the Django project's settings and models.
- **makemigrations**: Generates new database migration files based on changes to the project's models.
- **migrate**: Applies database migrations to create or modify tables in the database.
- **test**: Runs the project's automated tests and reports the results.

Django “Hello World”

Build the simplest Django project

Set-up the project

Create a new folder for the project:

mkdir helloworld

Move inside the folder with:

cd helloworld

Activate the anaconda environment with Django installed:

conda activate django

Start a new Django project:

django-admin startproject django_project .

Run the Django web server:

python manage.py runserver

To solve the **migration problem** run:

python manage.py migrate

It creates a SQLite database and migrated its built-in apps. This is represented by the new file **db.sqlite3**

Create An App

In Django, an app is a self-contained module that **provides a specific set of functionality.** A single top-level Django project can contain multiple apps.

Django uses apps for:

- **Encapsulation**: Keep functionality of a project into smaller, self-contained apps, you can better encapsulate the code and make it more modular.
- **Reusability**: apps that are independent of any particular project can be reused in other projects.
- **Scalability**: Apps allow you to add new functionality to your project as needed, without having to modify the core codebase.

An app called “pages” is created with:

```
python manage.py startapp pages
```

A new folder called **pages** is created with the associated files.

```
├── pages
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── migrations
│   │   └── __init__.py
│   ├── models.py
│   ├── tests.py
│   └── views.py
```

App files

- **admin.py** is a configuration file for the built-in Django Admin app.
- **apps.py** is a configuration file for the app itself.
- **models.py** is where we define our database models which Django automatically translates into database tables.
- **migrations/** folder keeps track of any changes to our models.py file so it stays in sync with our database.
- **tests.py** is for app-specific tests.
- **views.py** is where we handle the *request/response* logic for our web app.

```
├── pages
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── migrations
│   │   └── __init__.py
│   ├── models.py
│   ├── tests.py
│   └── views.py
```

Add an app to the Django project

Even though our new app exists within the Django project, Django doesn't "know" about it until we explicitly add it to the project **settings.py** file.

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'pages.apps.PagesConfig', #new added app
]
```

PagesConfig is the name of the class within the **pages/apps.py** file.

```
from django.apps import AppConfig


class PagesConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'pages'
```

Create a basic View

A **view** is responsible for:

1. Processing the request
2. Accessing the necessary data from the database or other sources
3. Returning a response to the client

`HttpResponse` objects

class `HttpResponse`

In contrast to `HttpRequest` objects, which are created automatically by Django, `HttpResponse` objects are your responsibility. Each view you write is responsible for instantiating, populating, and returning an `HttpResponse`.

The `HttpResponse` class lives in the `django.http` module.

We will use the basic **HttpResponse** class provided directly by Django.

```
from django.http import HttpResponse

# Create your views here.

def homePageView(request):
    return HttpResponse("<h1>Hello World</h1>") # string of HTML code
```

Configure the app URLs

We create a new file called **urls.py** within the pages app.
It will manage all the URLs inside the specific app.

Our URL pattern has three parts:

- A Python regular expression for the empty string " "
- A reference to the view function called homePageView
- An optional name URL pattern called "home"

```
from django.urls import path
from .views import homePageView

urlpatterns = [
path("", homePageView, name="home"),
]
```

Include app URLs to project URLs

```
from django.contrib import admin
from django.urls import path, include # import include function

urlpatterns = [
    path('admin/', admin.site.urls),
    path("", include("pages.urls")), # included pages app URLS
]
```

Now whenever a user visits the homepage “ “, they will first be routed to the pages app and then to the **homePageView** view set in the **pages/urls.py** file.

Hello World

After lunching the server with: python manage.py runserver we got this new homepage.

Hello World